Universidad Mariano Gálvez de Guatemala

Campus Cobán, Alta Verapaz

Ingeniería en Sistemas



Estudiante: JOB JUAN JOSE JUAREZ CAAL.

Carnet: 0902-22-12946

CURSO: Base de datos 1

Sección: "A"

Cobán, noviembre de 2026

Enlace para git hub https://github.com/JobJJCaal32/Proyecto-Base-de-datos-1--Credi-Track.git

Contenido

1. Introducción 3
1.1 Tecnologías 3
1.2 Archivos de la base de datos 3
2. El modelo de datos 4
2.1 Visión general 4
2.2 Decisiones de diseño 5
2.3 Integridad y normalización 6
3. Cómo pensamos las consultas 7
3.1 Qué vive en la base de datos y qué en el back end 7
3.2 Procedimientos almacenados 7
3.3 Vistas 8
3.4 Índices 8
3.5 Transacciones y concurrencia 9
3.6 Verificación de consistencia 10
4. Seguridad 11
4.1 Usuarios de MySQL con mínimo privilegio 11
4.2 Autenticación 11
4.3 Autorización 12
4.4 Cifrado de datos sensibles 12
4.5 Auditoría y trazabilidad 12
4.6 Otras protecciones 13
5. Pruebas 13
6. Conclusiones y trabajo futuro 14

Si el índice aparece vacío, en Word haga clic derecho sobre él y elija Actualizar campos.

1. Introducción

CrediTrack es un sistema bancario desarrollado como proyecto final del curso Base de Datos I. Su núcleo es una banca virtual: los clientes consultan sus saldos, transfieren dinero, revisan sus movimientos y generan su estado de cuenta. Alrededor de ella se construyó la operación del banco que la hace posible: el registro de clientes y la apertura de cuentas, las cajas de ventanilla, la supervisión de operaciones y la auditoría.

El desarrollo partió de la base de datos. Primero se diseñó el modelo completo, con sus reglas de integridad, y después se construyeron el back end y el front end sobre él. La estructura de la base de datos no cambió durante el desarrollo de la aplicación.

Este documento explica tres cosas: cómo está pensado el modelo de datos, cómo se diseñaron las consultas y qué medidas de seguridad protegen la información. El detalle de cada campo está en el diccionario de datos, y el diagrama completo en el modelo entidad-relación.

1.1 Tecnologías

Capa	Tecnología	Uso en el proyecto
Base de datos	MySQL 8.0, motor InnoDB	Tablas, llaves, restricciones CHECK, triggers, procedimientos, vistas e índices. Juego de caracteres utf8mb4 con intercalación utf8mb4_spanish_ci
Back end	Java 21, Spring Boot 4.1.2, Spring Security 7	Reglas de negocio, transacciones, autenticación y autorización
Acceso a datos	Spring Data JPA (Hibernate 7) y JdbcTemplate	JPA para las entidades y las escrituras; JdbcTemplate para procedimientos, vistas y bloqueos
Front end	Thymeleaf, HTML y CSS	Pantallas de cada rol, servidas por el mismo proyecto
Herramientas	MySQL Workbench, IntelliJ IDEA, Maven	Diseño del modelo, desarrollo y construcción

1.2 Archivos de la base de datos

La base de datos se crea desde cero con siete scripts que se ejecutan en orden:

Script	Contenido
01_estructura.sql	Base de datos credi_track y sus 28 tablas, con llaves primarias, foráneas, UNIQUE y CHECK
02_triggers.sql	Los 44 triggers de validación, protección y auditoría
03_procedimientos.sql	Los 5 procedimientos almacenados
04_vistas.sql	Las 5 vistas de consulta
05_indices.sql	Los 8 índices adicionales
06_usuarios.sql	Los 3 roles y usuarios de MySQL con sus permisos

Script	Contenido
07_datos_semilla.sql	Catálogos, roles con sus permisos, el usuario SISTEMA, una sucursal y un empleado de cada rol

2. El modelo de datos

2.1 Visión general

El modelo tiene 28 tablas agrupadas en ocho módulos. Cada módulo responde a una parte del negocio y se puede leer por separado.

Módulo	Tablas	Para qué sirve
Catálogos	genero, departamento, municipio, tipo_cuenta, estado_cuenta, tipo_operacion, estado_transaccion, tipo_alerta	Valores fijos que referencian las demás tablas
Personas	direccion, identidad	Datos personales, guardados una sola vez por persona
Clientes y empleados	cliente, empleado, sucursal	Quién es cliente, quién trabaja en el banco y dónde
Acceso	rol, permiso, rol_permiso, usuario	Quién puede entrar al sistema y qué puede hacer
Sesiones	sesion_usuario, intento_acceso	Cada inicio de sesión y cada intento, exitoso o no
Dinero	cuenta, transaccion, movimiento_cuenta	Cuentas, operaciones y su efecto en los saldos
Caja y supervisión	sesion_caja, autorizacion, alerta	Turnos de ventanilla, aprobaciones y operaciones sospechosas
Auditoría	estado_cuenta_generado, bitacora_auditoria, bitacora_consulta	Rastro de cambios y de consultas

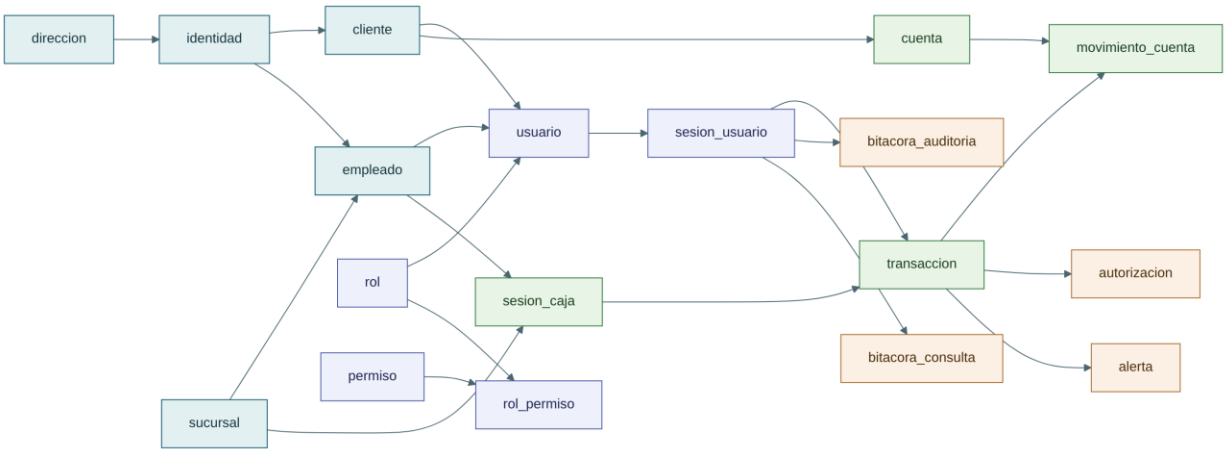


Figura 1. Relaciones principales entre las tablas. Las flechas van de la tabla referenciada a la que la referencia (relación de uno a muchos). Se omiten los catálogos.

2.2 Decisiones de diseño

Una sola identidad por persona

Los datos personales (DPI, nombres, fecha de nacimiento, teléfono y dirección) viven en **identidad**. Cliente y empleado son especializaciones de ella: cada una tiene su propia llave primaria y una columna `id_identidad` que es a la vez UNIQUE y llave foránea. Así una persona se registra una sola vez, el DPI no se puede repetir, y el cliente y el empleado conservan sus propios códigos, estados y fechas sin depender uno del otro.

El usuario como arco de tres vías

Todo acceso al sistema es una fila de **usuario**, que puede pertenecer a un cliente, a un empleado o ser el usuario interno SISTEMA, que hace la carga inicial. La columna `tipo` indica cuál de los tres es, y la restricción `chk_usuario_arco` obliga a que solo esté lleno el campo que corresponde: `id_cliente` si es CLIENTE, `id_empleado` si es EMPLEADO y ninguno si es SISTEMA. Ambos campos son UNIQUE, de modo que un cliente o un empleado tiene como máximo un usuario.

Coherencia entre el rol y el tipo de usuario

Un cliente nunca debe recibir un rol de empleado. Para garantizarlo en la base de datos y no solo en la aplicación, rol tiene la columna `tipo_usuario` y un UNIQUE sobre (`id_rol`, `tipo_usuario`). Usuario apunta a rol con una llave foránea compuesta (`id_rol`, `tipo`) → rol (`id_rol`, `tipo_usuario`). Si alguien intenta asignar el rol CAJERO a un usuario de tipo CLIENTE, la base lo rechaza.

Quién registra a quién

Cliente y empleado guardan qué usuario los registró, con una llave foránea compuesta (`id_usuario_registro`, `tipo_registro`) → usuario (`id_usuario`, `tipo`) y un CHECK que solo admite EMPLEADO o SISTEMA en `tipo_registro`. Así un cliente no puede aparecer como registrado por otro cliente. Como usuario también apunta a cliente y a empleado, estas llaves forman un ciclo; se resolvió creando las tablas primero y agregando esas llaves al final con ALTER TABLE. La misma técnica se usó entre sucursal y empleado (el responsable de una sucursal es un empleado y cada empleado trabaja en una sucursal).

Transacción y movimientos: la partida doble

Una operación de dinero se guarda en dos niveles. **transaccion** es el hecho: qué operación fue, por cuánto, cuándo, por qué canal y quién la hizo. **movimiento_cuenta** es su efecto en cada cuenta: un depósito genera un abono, un retiro un cargo y una transferencia un cargo en la cuenta de origen y un abono en la de destino. Cada movimiento guarda el saldo anterior y el posterior, y un CHECK verifica la aritmética (un abono suma y un cargo resta). El UNIQUE sobre (`id_transaccion`, `orden`) limita a dos los movimientos de una transacción.

Con este diseño, el historial de cualquier cuenta se reconstruye completo y el saldo de una cuenta siempre se puede comprobar contra sus movimientos.

El saldo protegido

El saldo de una cuenta es el dato más delicado del sistema, así que la base de datos no permite cambiarlo directamente. Una cuenta se crea con saldo cero (trigger `trg_cuenta_bi`); el depósito de apertura entra

como una transacción con canal APERTURA. Después, el trigger `trg_cuenta_bu` rechaza cualquier cambio de saldo, salvo que la variable de sesión `@permitir_cambio_saldo` valga 1. La aplicación activa esa variable en un solo lugar, justo después de registrar el movimiento, y la apaga inmediatamente después del UPDATE. Un UPDATE escrito a mano desde Workbench es rechazado con el mensaje «El saldo solo se modifica dentro de una transacción de dinero».

Nada se borra

En un banco el historial es evidencia. Diecisiete tablas tienen un trigger BEFORE DELETE que rechaza el borrado, y todas las llaves foráneas usan ON DELETE RESTRICT. En lugar de borrar, los registros cambian de estado: un cliente pasa a INACTIVO con su fecha de baja, una cuenta a CERRADA, un usuario a INACTIVO. Además, cuatro tablas son inmutables y tampoco aceptan UPDATE: movimiento_cuenta, intento_acceso, bitacora_auditoria y bitacora_consulta. De una transacción solo puede cambiar el estado.

Códigos visibles separados de las llaves

Las llaves primarias son números AUTO_INCREMENT que el usuario nunca ve. Lo que se muestra en pantallas y comprobantes son códigos con formato propio, todos con UNIQUE: CLI-000001 para clientes, EMP-0001 para empleados, AG-001 para sucursales, TRX-0000000001 para transacciones y números de cuenta de diez dígitos desde 1000000001. Así el formato de un código puede cambiar sin tocar ninguna relación.

Tipos de datos

- El dinero usa DECIMAL(15,2), nunca tipos de punto flotante, para que no haya errores de redondeo.
- Las fechas con hora se guardan como DATETIME en hora UTC; la aplicación las convierte a hora de Guatemala al mostrarlas.
- El DPI y el teléfono se guardan cifrados en hexadecimal, por eso son VARCHAR(255).
- La bitácora guarda los valores anteriores y nuevos como JSON, lo que permite registrar cambios de cualquier tabla con la misma estructura.
- sesion_caja usa una columna generada (`cajero_abierto`) que vale el id del cajero solo mientras la caja está abierta. Con un UNIQUE sobre ella, un cajero no puede tener dos cajas abiertas a la vez, algo que un UNIQUE normal no puede expresar.

2.3 Integridad y normalización

La integridad se aplica en la base de datos, no solo en la aplicación, para que se cumpla aunque alguien se conecte por otro medio.

Mecanismo	Cantidad	Qué garantiza
Llaves primarias	28	Cada fila se identifica de forma única
Llaves foráneas	47	No hay referencias a registros inexistentes; todas con ON DELETE y ON UPDATE RESTRICT
Restricciones UNIQUE	29	DPI, códigos, usuarios, correos y combinaciones que no se pueden repetir

Mecanismo	Cantidad	Qué garantiza
Restricciones CHECK	46	Valores permitidos, coherencia entre columnas y aritmética de los movimientos
Triggers de validación	6	Edad mínima de 18 años, saldo protegido, reglas de cierre de cuenta y transacciones que no se alteran
Triggers de inmutabilidad	4	Movimientos, intentos de acceso y bitácoras no se modifican
Triggers contra borrado	17	Ningún registro importante se elimina
Triggers de auditoría	17	Cada alta y cambio en las tablas sensibles queda en la bitácora

El modelo está en tercera forma normal: los catálogos están en tablas propias, la dirección tiene su propia tabla y cada dato depende solo de la llave de su tabla. Hay tres redundancias controladas, mantenidas a propósito:

- **saldo_actual** en cuenta se podría calcular sumando los movimientos, pero se guarda para consultarlo rápido y para bloquear la fila durante una operación. La pantalla de verificación de consistencia comprueba que siempre coincida con la suma de los movimientos.
- **saldo_anterior y saldo_posterior** en cada movimiento son una fotografía histórica: muestran el saldo exacto en ese momento sin recalcular toda la historia.
- **username** en las bitácoras se copia como texto para que el registro siga siendo legible aunque el usuario cambie o se desactive.

3. Cómo pensamos las consultas

3.1 Qué vive en la base de datos y qué en el back end

Una de las decisiones más importantes fue qué poner en la base de datos y qué dejar en la aplicación. El criterio fue este:

- **En la base de datos** queda lo que debe cumplirse sin importar quién se conecte: restricciones, triggers de protección y auditoría, y las operaciones de sesión que deben ser atómicas.
- **En el back end** queda la lógica de negocio que puede cambiar: el límite para retirar sin autorización (Q 10,000), los montos mínimos de cada operación, el flujo de aprobación de un supervisor o la evaluación de alertas. Estas reglas viven en servicios de Java con transacciones (@Transactional) y se configuran sin tocar la base.

Por el mismo criterio se descartaron las funciones almacenadas que se habían planeado (fn_tiene_permiso, fn_nombre_completo y fn_saldo_disponible): su lógica era más clara y más fácil de probar en la aplicación.

3.2 Procedimientos almacenados

Se usaron procedimientos donde varias operaciones tenían que pasar juntas y protegidas, sin depender de que la aplicación las haga en el orden correcto.

Procedimiento	Qué hace
sp_registrar_login	Registra cada intento de inicio de sesión. Si es correcto, reinicia el contador de fallos, genera un token aleatorio de 64 caracteres (RANDOM_BYTES) y abre la sesión. Si falla por contraseña, suma el intento y al tercero bloquea al usuario y cierra sus sesiones abiertas
sp_validar_sesion	Se llama en cada petición. Verifica que el token exista, que la sesión no esté cerrada, que no lleve más de 15 minutos sin uso y que el usuario siga activo. Si todo está bien, actualiza la hora de último uso y fija las variables de bitácora
sp_cerrar_sesion	Cierra la sesión del token con su motivo (por defecto LOGOUT)
sp_consultar_cuentas	Devuelve las cuentas y saldos de un cliente. Si quien consulta es un cliente y pide cuentas ajenas, registra el intento como ACCESO DENEGADO y lo rechaza. Toda consulta queda en bitacora_consulta
sp_movimientos	Devuelve los movimientos de un cliente o una cuenta, filtrados por fechas y tipo de operación, con las mismas reglas de acceso y registro que el anterior

3.3 Vistas

Las vistas juntan en una sola consulta lo que de otro modo se repetiría en muchos lugares de la aplicación. También sirven de frontera de seguridad: el usuario de auditoría de MySQL puede leer las vistas sin tener acceso a las columnas sensibles de las tablas.

Vista	Qué muestra	Dónde se usa
vw_cuentas_cliente	Cada cuenta con su titular, tipo, estado y saldo disponible (saldo menos retenido)	Mis cuentas, ficha del cliente, sp_consultar_cuentas
vw_movimientos	Cada movimiento con su transacción, la cuenta contraparte y quién la hizo	Movimientos del cliente y del auditor, comprobantes
vw_bitacora_legible	La bitácora con el tipo de usuario, la IP y el equipo, y la acción en palabras	Pantalla de bitácora del auditor
vw_intentos_fallidos	Los intentos fallidos con el estado actual del usuario intentado	Pantalla de intentos de acceso
vw_operaciones_sucursal	Cada transacción con su sucursal y el empleado que la hizo, o «Banca virtual»	Reporte de operaciones por sucursal o cajero

3.4 Índices

Además de los índices que MySQL crea solo para las llaves primarias, foráneas y UNIQUE, se agregaron ocho índices pensados para las consultas que más se repiten:

Índice	Columnas	Consulta que acelera
idx_transaccion_fecha	transaccion (fecha_hora)	Movimientos y reportes por periodo
idx_intento_acceso_usuario_fecha	intento_acceso (username_intentado, fecha_hora)	Intentos fallidos de un usuario en un rango de fechas

Índice	Columnas	Consulta que acelera
idx_intento_acceso_ip_fecha	intento_acceso (direccion_ip, fecha_hora)	Intentos desde una misma IP
idx_bitacora_auditoria_registro	bitacora_auditoria (nombre_tabla, id_registro_afectado)	Historial completo de un registro
idx_bitacora_auditoria_fecha	bitacora_auditoria (fecha_hora)	Bitácora de un periodo
idx_bitacora_consulta_fecha	bitacora_consulta (fecha_hora)	Consultas de clientes de un periodo
idx_alerta_estado	alerta (estado)	Alertas pendientes del supervisor
idx_identidad_nombre	identidad (primer_apellido, primer_nombre)	Búsqueda de personas por nombre

En los índices compuestos, la columna de igualdad va primero y la fecha después, para que MySQL encuentre el grupo y recorra solo el rango de fechas pedido.

3.5 Transacciones y concurrencia

El riesgo más serio en un sistema bancario es que dos operaciones simultáneas sobre la misma cuenta dejen un saldo equivocado. Por eso cada operación de dinero se ejecuta dentro de una sola transacción de base de datos y sigue siempre los mismos pasos. Así ocurre una transferencia:

1. Se validan los datos y la clave de operación del cliente.
2. Se bloquean las dos cuentas con SELECT … FOR UPDATE, siempre en orden de id. Si dos transferencias cruzadas llegan al mismo tiempo, ambas piden los bloqueos en el mismo orden y no se produce un interbloqueo.
3. Se registra la transacción con un número correlativo, generado también con un bloqueo para que no se repita.
4. Por cada cuenta se lee el saldo bloqueado, se verifica que la cuenta permita operar y que haya saldo disponible, se inserta el movimiento con su saldo anterior y posterior, y se actualiza el saldo.
5. Se evalúan las alertas (monto alto y operaciones frecuentes) y se confirma todo. Si cualquier paso falla, se deshace la operación completa.

Este es el núcleo del paso 4, tal como está en el servicio que mueve los saldos:

```
SELECT c.numero_cuenta, c.saldo_actual, c.saldo_retenido, ec.permite_operar
  FROM cuenta c
  JOIN estado_cuenta ec ON ec.id_estado_cuenta = c.id_estado_cuenta
 WHERE c.id_cuenta = ?
   FOR UPDATE OF c;

INSERT INTO movimiento_cuenta (...) VALUES (...);
SET @permitir_cambio_saldo = 1;
UPDATE cuenta SET saldo_actual = ? WHERE id_cuenta = ?;
SET @permitir_cambio_saldo = 0;
```

Dos medidas más completan la protección:

- **Sin operaciones duplicadas.** Cada formulario lleva una referencia única que se guarda en la columna `referencia` de transaccion, con UNIQUE. Si el usuario presiona dos veces el botón o recarga la página, la segunda vez se encuentra la misma referencia y no se cobra otra vez.
- **Reintento ante bloqueos.** Si MySQL detecta un interbloqueo o un choque al generar un código, la aplicación repite la transacción completa hasta cinco veces antes de mostrar un error.

En las pruebas se lanzaron 20 transferencias simultáneas entre dos cuentas: todas terminaron bien y los saldos finales fueron exactos.

3.6 Verificación de consistencia

Para el auditor se construyó una pantalla que revisa toda la base de datos con siete consultas y muestra cuántos casos no cumplen cada regla. Si todo está bien, el resultado es cero en todas.

Comprobación	Regla que verifica
Saldos de las cuentas	El saldo de cada cuenta es igual a la suma de sus abonos menos sus cargos
Cadena de movimientos	Cada movimiento empieza con el saldo en que terminó el anterior de la misma cuenta
Movimientos de cada transacción	Cada transacción aplicada tiene exactamente los movimientos que le corresponden, por su monto
Reversos	Cada reverso anula una transferencia REVERSADA, por el mismo monto y con las cuentas invertidas
Cuentas cerradas	Una cuenta cerrada no tiene saldo y tiene fecha de cierre
Cierre de cajas	El efectivo esperado de cada caja es apertura más depósitos menos retiros
Rastro en la bitácora	Cada transacción y cada cuenta tienen su registro de creación en la bitácora

La primera comprobación se resuelve con esta consulta; cualquier fila que devuelva es una cuenta descuadrada:

```
SELECT c.numero_cuenta, c.saldo_actual, COALESCE(m.total, 0) AS suma_movimientos
  FROM cuenta c
  LEFT JOIN (SELECT id_cuenta,
                    SUM(IF(naturaleza = 'ABONO', monto, -monto)) AS total
               FROM movimiento_cuenta
              GROUP BY id_cuenta) m ON m.id_cuenta = c.id_cuenta
 WHERE c.saldo_actual <> COALESCE(m.total, 0);
```

La cadena de movimientos usa la función de ventana LAG de MySQL 8 para comparar cada movimiento con el anterior de su cuenta. En las pruebas se alteró a mano un saldo y una caja desde MySQL, y la pantalla detectó ambos casos.

4. Seguridad

La seguridad se diseñó en capas: si una falla, la siguiente sigue protegiendo los datos. Cada capa se describe a continuación, desde la base de datos hacia la aplicación.

4.1 Usuarios de MySQL con mínimo privilegio

La aplicación no se conecta con el usuario root. Se crearon tres roles de MySQL y un usuario para cada uno, y cada rol recibe solo los permisos que necesita:

Usuario	Para qué	Permisos
ct_backend	La aplicación	SELECT en catálogos; SELECT, INSERT y UPDATE en las tablas que la aplicación modifica; solo SELECT e INSERT en movimientos, autorizaciones y bitácora de consultas; solo SELECT en la bitácora de auditoría. DELETE únicamente en rol_permiso. EXECUTE en los 5 procedimientos
ct_auditor	Revisión y auditoría	Solo lectura. En usuario no ve el correo ni los hashes; en identidad no ve el DPI, el NIT ni el teléfono (permisos por columna)
ct_admin	Mantenimiento	Todos los privilegios sobre credi_track, para el administrador de la base de datos

Así, aunque alguien tomara el control de la aplicación, no podría borrar transacciones ni modificar la bitácora: el usuario con que se conecta no tiene esos permisos, y además los triggers lo impiden. Los tres usuarios se bloquean un día tras cinco contraseñas incorrectas (FAILED_LOGIN_ATTEMPTS 5, PASSWORD_LOCK_TIME 1), y la contraseña de ct_backend no está escrita en el código: se lee de una variable de entorno o del archivo de configuración local, que no se sube al repositorio.

```
GRANT SELECT (id_usuario, tipo, id_cliente, id_empleado, id_rol, username, ...)
    ON credi_track.usuario TO rol_ct_auditor;
GRANT SELECT (id_identidad, primer_nombre, primer_apellido, fecha_nacimiento, ...)
    ON credi_track.identidad TO rol_ct_auditor;
```

4.2 Autenticación

- **Contraseñas con BCrypt.** Se guardan solo como hash BCrypt (costo 10), nunca en texto plano ni en la bitácora. La clave de operación para transferir se guarda igual.
- **Política de contraseñas.** Entre 8 y 64 caracteres, con mayúscula, minúscula, número y símbolo, y sin contener el nombre de usuario.
- **Contraseña temporal.** Al registrar un cliente o un empleado se genera una contraseña aleatoria de 10 caracteres que debe cambiar en su primer ingreso; mientras no la cambie, no puede usar ninguna otra pantalla.
- **Bloqueo por intentos.** Al tercer intento fallido el usuario queda BLOQUEADO y sus sesiones abiertas se cierran. Solo un administrador lo desbloquea.
- **Sesión en la base de datos.** El token de 64 caracteres viaja en una cookie HttpOnly (JavaScript no puede leerla), SameSite=Strict y Secure en producción (solo viaja por HTTPS). Se valida contra

sesion_usuario en cada petición, así que cerrar sesión o bloquear al usuario lo invalida al instante. Tras 15 minutos sin uso la sesión expira.

- **Protección CSRF.** Cada formulario lleva un token que impide que otra página envíe operaciones en nombre del usuario.

4.3 Autorización

Hay 8 roles y 33 permisos. Cada pantalla de la aplicación exige un permiso concreto, y el menú de cada usuario solo muestra lo que su rol permite. Si alguien escribe a mano la dirección de una pantalla que no le corresponde, recibe un error 403. Los permisos de un rol se pueden cambiar desde la aplicación y el cambio aplica en la siguiente página que abra el usuario. Además:

- **Cada cliente solo ve lo suyo.** Los procedimientos verifican que las cuentas consultadas sean del cliente de la sesión; un intento de ver cuentas ajenas se rechaza y queda registrado.
- **Separación de funciones.** Un retiro mayor a Q 10,000 queda pendiente hasta que un supervisor lo aprueba. El supervisor no puede aprobar operaciones de su propia caja ni arquearla (regla de cuatro ojos, reforzada con un CHECK en sesion_caja).
- **El administrador no puede quitarse el control.** No puede quitarle a su propio rol el permiso de asignar permisos, ni cambiar su propio rol ni darse de baja.

4.4 Cifrado de datos sensibles

El DPI y el teléfono se cifran con AES de 128 bits antes de guardarse. La aplicación usa exactamente el mismo algoritmo que la función AES_ENCRYPT de MySQL, así que un dato cifrado por la aplicación se puede descifrar desde un script y al revés; una prueba automática lo verifica. La clave tampoco está en el código: se lee de una variable de entorno o del archivo de configuración local.

El modo de cifrado elegido produce siempre el mismo resultado para el mismo DPI. Fue una decisión consciente: permite mantener el UNIQUE sobre el DPI cifrado y buscar a un cliente por su DPI sin descifrar toda la tabla. A cambio, dos DPI iguales se ven iguales cifrados, lo cual no es un riesgo porque el DPI no se repite.

En pantallas y bitácoras el DPI aparece oculto, por ejemplo *********6789. El auditor tampoco puede leerlo desde MySQL, por los permisos por columna.

4.5 Auditoría y trazabilidad

Cada cambio en las tablas sensibles (cliente, empleado, usuario, cuenta, transaccion, rol_permiso, sesion_caja, autorizacion y alerta) queda en **bitacora_auditoria** con los valores anteriores y nuevos. Los registran triggers, no la aplicación, así que no hay forma de olvidar registrar un cambio.

Para que los triggers sepan quién hizo el cambio, la aplicación fija dos variables de MySQL cada vez que toma una conexión del pool:

```
SET @usuario_app = ?, @id_sesion_usuario = ?, @permitir_cambio_saldo = 0;
```

Si el cambio se hace directamente en MySQL, fuera de la aplicación, la bitácora registra el usuario de MySQL que lo hizo. Además:

- **bitacora_consulta** registra quién consultó qué cliente: búsquedas por DPI, datos personales, saldos, movimientos y estados de cuenta.
- **intento_acceso** registra cada intento de inicio de sesión con su IP, aunque el usuario no exista.
- Las bitácoras y los intentos no se pueden modificar ni borrar, ni siquiera con permisos de escritura.
- El auditor tiene pantallas para consultar la bitácora, los intentos fallidos, las consultas de clientes, las cajas con faltantes o sobrantes y la verificación de consistencia.

4.6 Otras protecciones

- **Inyección SQL.** Todas las consultas usan parámetros (? en JdbcTemplate y consultas preparadas en JPA); lo que escribe el usuario nunca se concatena al SQL.
- **Scripts en pantalla (XSS).** Thymeleaf escapa todo el texto que muestra, así que un nombre con código HTML se ve como texto y no se ejecuta.
- **Mensajes de error.** Los errores de reglas de negocio, como los de los triggers, se muestran al usuario tal cual. Cualquier otro error muestra un mensaje genérico y el detalle queda solo en el registro del servidor, sin revelar la estructura de la base.

5. Pruebas

Cada parte se probó desde cero antes de darla por terminada. En total se aprobaron 366 pruebas automáticas:

Nivel	Pruebas	Qué cubren
Base de datos	75	Scripts desde cero, restricciones, triggers, procedimientos y vistas
Aplicación por HTTP	275	Cada pantalla y operación de los 6 roles, en 8 grupos: inicio de sesión y bloqueo, permisos y 403, registro de clientes, transferencias, caja, supervisión, administración y auditoría
Unitarias de Java	16	Cifrado AES idéntico al de MySQL, política de contraseñas, contraseñas temporales y fechas en hora de Guatemala

Algunas de las pruebas más importantes:

- Un registro de cliente que falla a la mitad no deja nada guardado.
- Veinte transferencias simultáneas terminan con los saldos exactos.
- Reenviar el mismo formulario no duplica una operación.
- Un cliente no puede ver cuentas, movimientos ni comprobantes de otro cliente.
- Un saldo alterado a mano en MySQL es detectado por la verificación de consistencia.

6. Conclusiones y trabajo futuro

El modelo cumplió su objetivo: la aplicación completa se construyó sobre él sin cambiar su estructura, y las reglas más importantes (saldos, historial, permisos y auditoría) se cumplen en la propia base de datos, sin depender de que la aplicación las respete.

Como trabajo futuro, la mejora principal sería separar el modelo en dos: una base con lo que corresponde a la banca virtual y otra con la operación interna del banco (cajas, sucursales y empleados). Podrían vivir en la misma base sin relaciones entre sus tablas, o en bases distintas conectadas por la lógica de la aplicación, por ejemplo con microservicios. Así cada parte podría crecer por separado.